

Bocconi

COMPUTER SCIENCE

Code 30424 a.y. 2024-2025

Lesson 2

Digital encoding



Università
Bocconi
MILANO

Objectives of the lesson

- Understand the nature of different data types
- Understand the binary logic underlying computers
- Understand the logic of binary encoding

The concept of Information

To formalize numeric and alphanumeric data, **sequences of symbols** chosen from a finite set called an **alphabet** are used

e h m o 3 5 9 1 4 6 + , ! . @ € &



The concept of Information

Each alphabet is associated with a set of composition rules that define **well-formed sequences**, i.e. those that allow **data** and **instructions** to be identified

e h m o 3 5 9 1 4 6 + , ! . @ € &



home



if X+1<=0:



3,591.46



The concept of Information

Well-formed sequences of symbols of an alphabet acquire meaning thanks to the use of **codes**

e h m o 3 5 9 1 4 6 + , ! . @ € &

home

if X+1<=0:

3,591.46

English language

Python language

Positional notation



Alphabets and sequences

In the case of sequences all consisting of the same number of elements, it is possible to determine the number of different sequences that can be created (i.e. the number of all possible combinations of symbols) based on the **number of symbols** present in the alphabet and the **length** of the sequences.

Namely:

given an alphabet of
 n symbols we can obtain

$$n^k$$

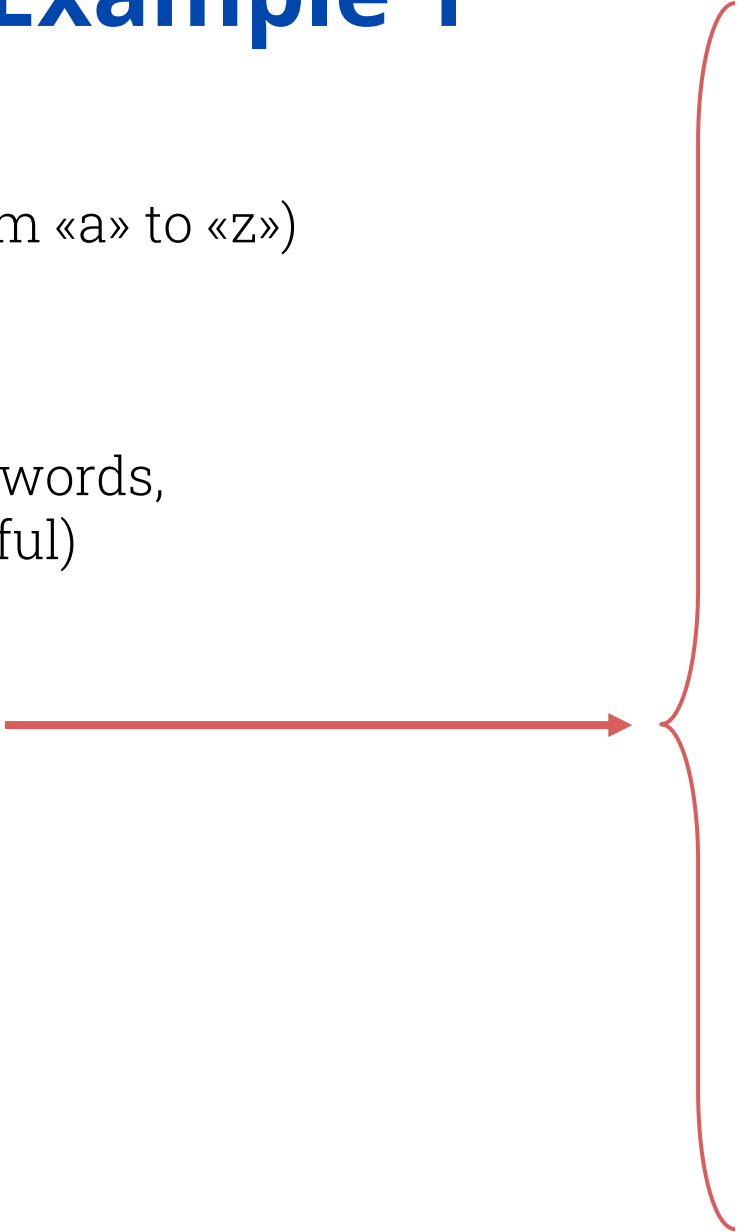
different sequences of length k

Alphabets and sequences: Example 1

Alphabet used: Anglo-Saxon
(26 symbols, from «a» to «z»)

Sequence length: 3 elements
(i.e. 3-character words,
not all meaningful)

N. of possible sequences: $26^3 = 17,576$



aaa
aab
aac
aad
...
aaw
aax
aay
aaz
aba
abb
abc
abd
...
...
...
...
...
...
zzw
zzx
zzy
zzz



Alphabets and sequences: Example 2

Alphabet used: decimal digits
(10 symbols, from «0» to «9»)

Sequence length: 3 elements
(i.e. 3-digit natural numbers)

N. of possible sequences: $10^3 = 1,000$



000
001
002
003
004
005
006
007
008
009
010
011
012
...
...
...
...
995
996
997
998
999



Alphabets and sequences: Example 3

Alphabet used: binary digits
(2 symbols, «0» and «1»)

Sequence length: 3 elements
(i.e. 3-digit binary numbers)

N. of possible sequences: $2^3 = 8$

000

001

010

011

100

101

110

111

Alphabets and sequences: Example 4

Alphabet used: binary digits
(2 symbols, «0» and «1»)

Sequence length: 8 elements
(i.e. 8-digit binary numbers)

N. of possible sequences: $2^8 = 256$



```
00000000
00000001
00000010
00000011
00000100
00000101
00000110
00000111
00001000
00001001
00001010
00001011
```

...

...

...

...

```
11111010
11111011
11111100
11111101
11111110
11111111
```



Binary systems

The development of information technologies in the electronic age (since the half of the last century) has been based on the use of "bistable" devices and components, i.e. capable of assuming and maintaining only two possible states:

- electrical voltage level: high / low
- magnetic polarization: positive / negative
- luminous flux: on / off

The alphabet of data that these devices are able to recognize consists of only 2 symbols, conventionally represented by the **binary digits** «0» and «1».

bit = **binary digit**

Binary systems

- In modern computing the flow of binary digits – which is the basis of the operation of all digital systems – is conventionally grouped into sequences of **8 bits**, commonly defined as **Bytes**
- A Byte can therefore represent $2^8 = \mathbf{256}$ different bit combinations
- In a computer both instructions and data (numbers, texts, images, sounds, videos, ...) are encoded with sequences of bits whose length is expressed in Bytes
- Consistently with the binary representation of data, Byte multiplication factors (Kilo, Mega, Giga, Tera, ...) have historically been expressed with the powers of 2 instead of 10 (therefore, instead of $10^3 = 1,000$ the nearest value in base 2 is used, i.e.: $2^{10} = 1,024$)

Measuring data in binary format

1 KiloByte (1 **KB**) = $[2^{10} \text{ Bytes}]$ = 1.024 Bytes

1 MegaByte (1 **MB**) = 1.024 KB = 1.048.576 Bytes

1 GigaByte (1 **GB**) = 1.024 MB = 1.073.741.824 Bytes

1 TeraByte (1 **TB**) = 1.024 GB = 1.099.511.627.776 Bytes



Binary encoding of natural numbers

- With a sequence of **n** bits we can represent 2^n natural numbers, from 0 to $2^n - 1$
- Using a **Byte** it is therefore possible to represent all **non-negative integers** from 0 to 255
- Each integer can be expressed with the corresponding binary value obtained with the mathematical process of conversion from **base₁₀** to **base₂**
- Considering the «**32-bit**» and «**64-bit**» architectures of nowadays computers (which operate in chunks of 32 bits = 4 Bytes or 64 bits = 8 Bytes) it is common to use more than 1 Byte to represent numbers



Binary encoding of natural numbers

Base ₁₀	Base ₂
000	0 0 0 0 0 0 0 0
001	0 0 0 0 0 0 0 1
002	0 0 0 0 0 0 1 0
003	0 0 0 0 0 0 1 1
004	0 0 0 0 0 1 0 0
005	0 0 0 0 0 1 0 1
006	0 0 0 0 0 1 1 0
007	0 0 0 0 0 1 1 1
008	0 0 0 0 1 0 0 0
...	
253	1 1 1 1 1 1 0 1
254	1 1 1 1 1 1 1 0
255	1 1 1 1 1 1 1 1



Example

Demonstration of how $153_{\text{dec}} = 10011001_{\text{bin}}$ using positional notation of decimal numbers and of binary numbers:

Decimal notation (with 3 digits)

10^2	10^1	10^0
100	10	1

1	5	3
---	---	---

1×100	5×10	3×1
100	50	3
$= 100 + 50 + 3$		

=

Binary notation (with 8 bits)

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

1	0	0	1	1	0	0	1
---	---	---	---	---	---	---	---

1×128	0×64	0×32	1×16	1×8	0×4	0×2	1×1
128	0	0	16	8	0	0	1
$= 128 + 16 + 8 + 1$							

Example of a sum

By using positional notation for numbers and considering how carryovers are handled in a sum:

$$\begin{array}{r}
 87 \\
 + 66 \\
 \hline
 = 153
 \end{array}
 \quad \longrightarrow \quad
 \begin{array}{r}
 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 1 \ 1 \\
 + 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \\
 \hline
 = 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1
 \end{array}$$

Carryovers	$0 + 0 = 0$
in base 2:	$0 + 1 = 1$
	$1 + 0 = 1$
	$1 + 1 = 0$ with a carryover of 1



Binary encoding of integer numbers

- To represent positive and negative integers, the first bit of the sequence is used for the sign («1» for negative, «0» for positive)
- Considering the type of data defined as «**Long**» that uses **4 Bytes** to represent **signed integers**, we have the following:
 - 4 Bytes = 32 bits
 - 1 bit is used for the sign means that we can have 2^{31} different sequences of bits (i.e. 2,147,483,648)
 - In this way the type «Long» can represent all integers from **-2,147,483,648** to **+2,147,483,647** (note: positive integers are one less than the negative ones because of the «0»)



Binary encoding of floating point numbers

- The most common method to represent floating point numbers (i.e. numbers with decimals) relies on **scientific notation**
- Scientific notation used in computers consider any floating point value as a decimal number with only 1 digit to the left of the separator, multiplied by a power of ten so to determine the correct position of the decimal separator
- For instance, the value **153.82** is expressed as **1.5382×10^2** , commonly written as **1.5382E+02** (where «1.5382» is called the **significand** and «2» the **exponent**)
- Using a «32-bit» floating point encoding: **1 bit** is used for the **sign**, **8 bits** are used for the **exponent** and **23 bits** for the **significand** (you can refer to the IEEE 754 standard for more details)

To summarize

Data type (with alternative names)	Storage size	Range
Byte	1 byte	0 to 255
Integer	2 bytes	-32,768 to 32,767
Long (Long integer)	4 bytes	-2,147,483,648 to 2,147,483,647
LongLong (LongLong integer)	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
Single (single-precision floating-point)	4 bytes	-3.402823E38 to -1.401298E-45 for negative values
Double (double-precision floating-point)	8 bytes	-1.79769313486231E308 to -4.94065645841247E-324 for negative values
Currency (scaled integer)	8 bytes	-922,337,203,685,477.5808 to 922,337,203,685,477.5807
Date	8 bytes	-657,434 (January 1, 100) to 2,958,465 (December 31, 9999)

(list of most relevant data types available in Excel)

Binary encoding of text

- With a sequence of **n** bits we can also represent **2^n** different **symbols**
- Using a **Byte** it is therefore possible to conventionally represent **256 alphanumeric symbols**, including all uppercase and lowercase letters, punctuation symbols, special characters, etc.
- To include all characters and symbols from different alphabets **2 or more Bytes** can be used ($2^{16} = 65.536$ symbols)
- Different standards (i.e. “**Tables of characters**”) are available to represent each symbol: ASCII, UNICODE, UTF-8, etc.

ASCII

- Acronym for **American Standard Code for Information Interchange**
- Developed starting from 1961 and lastly updated in 1986
- First release was based on **7-bit** = 128 symbols
- Last release (commonly known as **Extended ASCII**) was based on **8-bits** = 256 symbols
- The set of symbols include printable ones and various control elements
- Even the extended version is barely enough to represent US English characters (therefore not suitable for universal use)

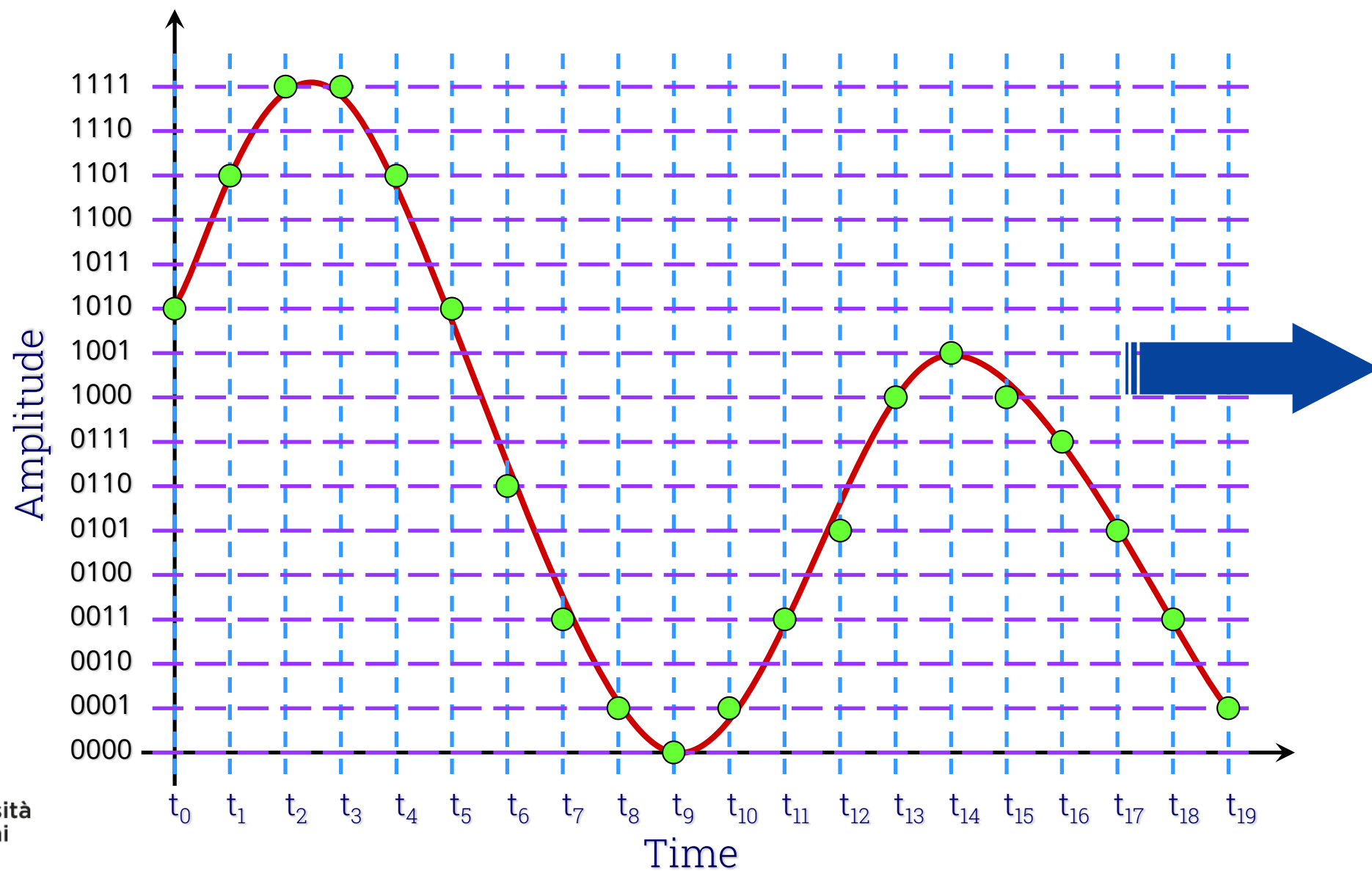
UNICODE and UTF-8

- UNICODE development begins in 1988 as an **international/multilingual text character encoding system**
- As of 2024 the standard defines **154,998 characters** and **168 scripts**
- **Scripts** are collections of written signs used to represent text in different writing systems
- The first 128 elements correspond to those of the ASCII code
- UTF-8 (Unicode Transformation Format – 8-bit) is a variable-length character encoding standard based on Unicode standard
- UTF-8 is the most common character encoding system for the World Wide Web

Digitization of multimedia contents

- The conversion of analog materials – such as images, video and audio – into a stream of bits requires several steps: sampling, quantization, encoding, compression, etc.
- **Sampling** is the process by which we divide an object into measurable fragments
- **Quantization** is the process of representing each sample with a finite set of discrete values, by assigning a numerical value in binary format
- **Encoding** and **compression** depend on the type of content, the platform used, the purposes of use of the digitized material, etc.

Binary encoding of sounds (example using 4 bits)



$t_0 = 1010$
 $t_1 = 1101$
 $t_2 = 1111$
 $t_3 = 1111$
 $t_4 = 1101$
 $t_5 = 1010$
 $t_6 = 0110$
 $t_7 = 0011$
 $t_8 = 0001$
 $t_9 = 0000$
 $t_{10} = 0001$
 $t_{11} = 0011$
 $t_{12} = 0101$
 $t_{13} = 1000$
 $t_{14} = 1001$
 $t_{15} = 1000$
 $t_{16} = 0111$
 $t_{17} = 0101$
 $t_{18} = 0011$
 $t_{19} = 0001$

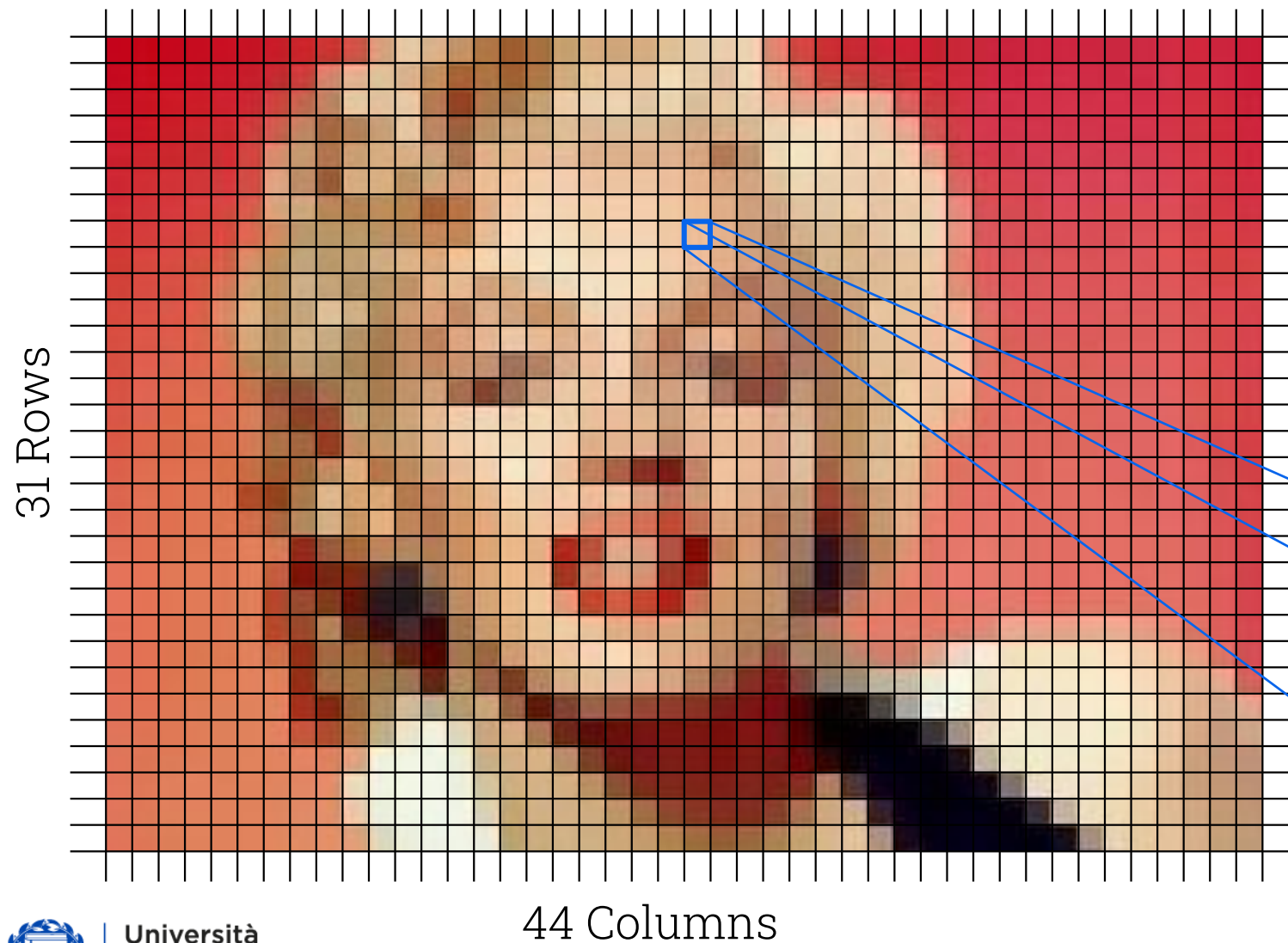
Binary encoding of sounds

- The number of samples acquired depends on the sampling frequency
- The sampling frequency for a good quality audible sound (CD quality) is **44.1 KHz** (44,100 samples per second)
- The set of discrete values to quantify each sample in CD quality requires **2 Bytes** (i.e. 16 bits = 65,536 discrete values)
- Therefore, to store **1 minute** of stereophonic music (not compressed) in **CD quality** we need:

$$44,100 \text{ samples/sec.} * 60 \text{ sec.} * 2 \text{ Bytes} * 2 \text{ channels (L/R)} = 10,584,000 \text{ Bytes}$$



Binary encoding of images



Sampling:

31 Rows * 44 Columns
= 1,364 pixels
= 0.001 MP (approx.)

Quantization:

RED = 11101011 (235)
GREEN = 10111011 (187)
BLUE = 10010011 (147)
Transparency = 00000000 (0)

Binary encoding of images

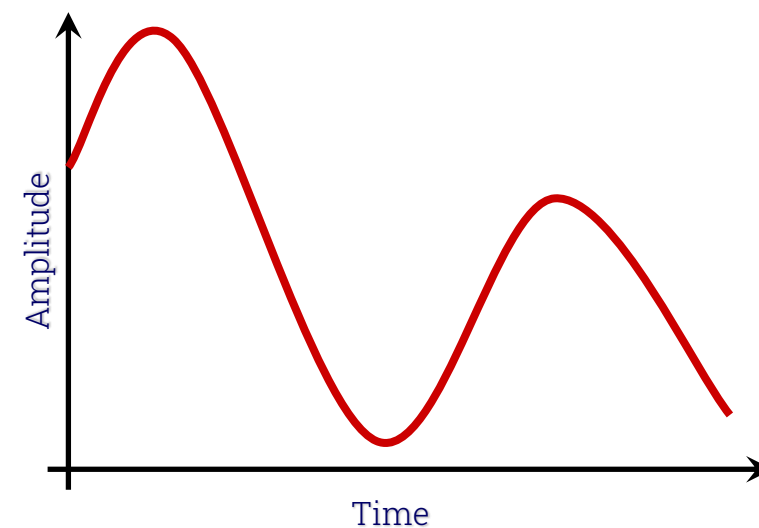
- The number of samples acquired correspond to the number of **pixels**
- The camera sensor of a smartphone can reach 200MP (200,000,000 pixels)
- The color depth of each pixel is commonly of **16,777,216 colors** (1 Byte for each RGB color component), with **256 levels of transparency** (1 Byte for the alpha channel) for a total of **4 Bytes** used for each pixel
- Therefore, to store a picture (not compressed) taken with a “Samsung S25 Edge” we need:

$$200,000,000 \text{ pixels} * 4 \text{ Bytes} = 800,000,000 \text{ Bytes} = 763 \text{ MB (approx.)}$$

Binary encoding of videos



Multichannel audio



FPS
(Frame per Second)



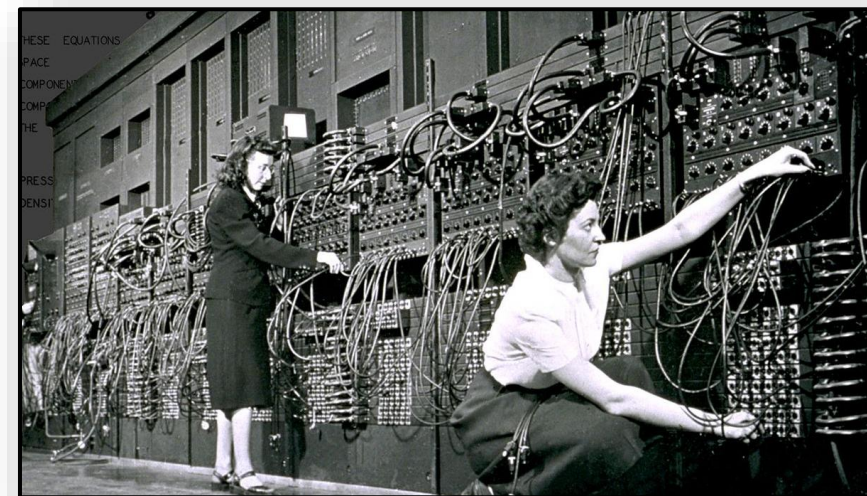
Binary encoding of videos

- The number of samples relates to the number of **frames per second**, the number of **pixels** in each frame and the number of **sound samples**
- The quantification of each pixel and sound sample follows the same principles described for images and sounds (see previous slides)
- The number of **frames per second** needed to perceive a smooth movement on the screen is between **30** and **120 fps**
- Therefore, to store 1 minute of **4K** video (8.3 millions of pixels per frame) in **HFR** (High Frame rate @ 120 fps) with **Dolby Atmos** sound (24 bits @ 192 KHz), before compression, we need:

238,878,720,000 Bytes of video + 69,120,000 Bytes of audio = 222.54 GB (approx.)

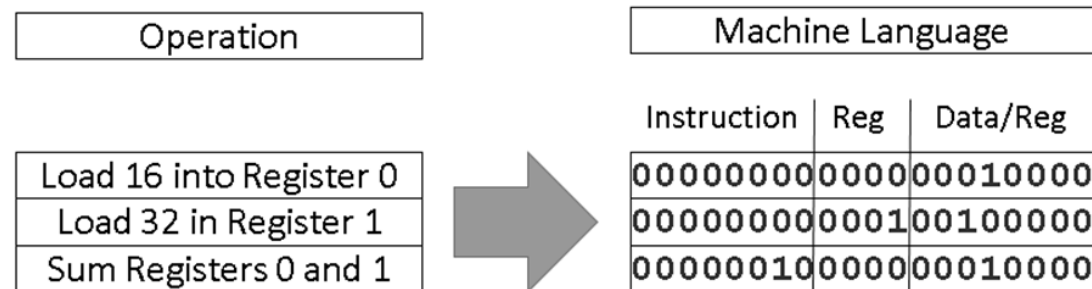
Binary encoding of instructions

- To make a computer process our data, we need to provide precise **instructions** and **commands**
- In the 1950s, instructions and commands were directly provided as long **sequences of bits** (i.e. strings of 0 and 1)
- Humans had to instruct the computer in so-called **Machine Language**
- Writing a very simple program required large amounts of time and extremely specialized technicians



Programming languages

- To facilitate **interaction** between **humans** and **machines**, programming languages have been developed
- A programming language is a **formal language** that enables to provide a computer with the instructions needed to solve a problem
- There are many programming languages, each with its own **syntax**, but basically all able to perform the same type of operations by sharing most of the logic and rules with each other

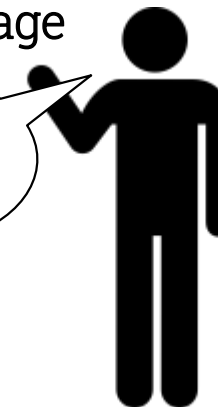


Evolution of programming languages

Going through decades, programming languages have evolved towards natural language and got far from 0 and 1 alone

Natural language

What is the result of 3 + 5 ?



High-level languages
(Java, C, Python, ...)

```
int main()
{
    int first, second, add, subtract;

    printf("Enter two integers\n");
    scanf("%d%d", &first, &second);

    add = first + second;
    subtract = first - second;

    printf("Sum = %d\n", add);
    printf("Difference = %d\n", subtract);

    return 0;
}
```

Assembly

A:	INT	16;
B:	INT	32;
LOAD	R0,	a;
LOAD	R1,	b;
ADD	R0,	R1;

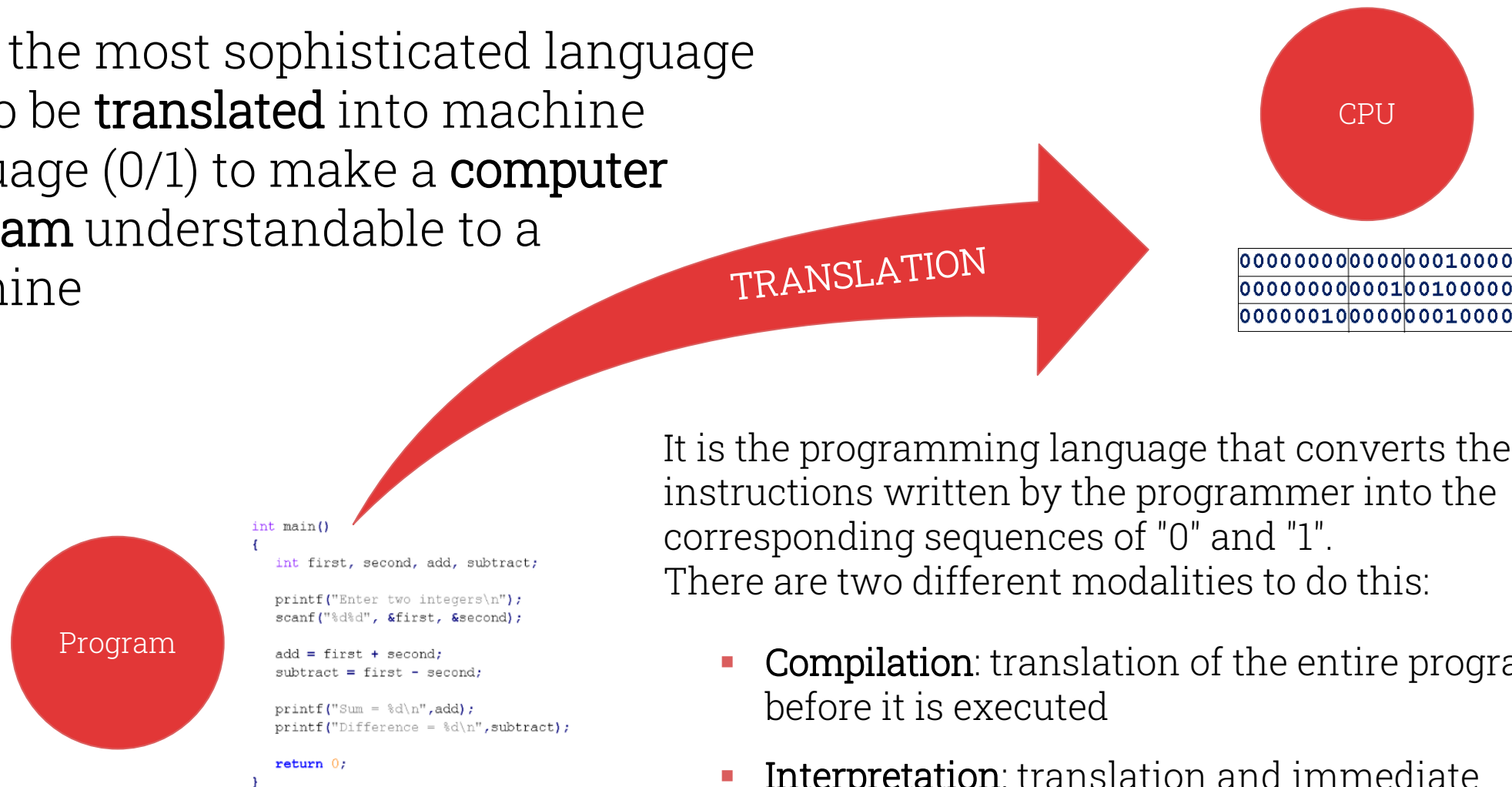
Machine Language

00000000	0000	00010000
00000000	0001	00100000
00000010	0000	00010000



Binary encoding of programs

Even the most sophisticated language has to be **translated** into machine language (0/1) to make a **computer program** understandable to a machine



It is the programming language that converts the instructions written by the programmer into the corresponding sequences of "0" and "1".

There are two different modalities to do this:

- **Compilation:** translation of the entire program before it is executed
- **Interpretation:** translation and immediate execution of single instructions

Knowledge to acquire with today's lesson

- Representation of data based on alphabets and sequences
- Basic elements of the binary system
- Numerical conversion between base 10 and base 2
- Data types and their digital representation
- Digital representation of analog media
- Encoding of instructions and commands

Study materials

This **set of slides** has to be considered the study material for the contents of **Lesson 2**

Any references to external resources have to be considered as supplementary material for further exploration of the topics covered